

NAMES: David Fatheree, Preston May

```
`timescale 1ns / 1ps
```

```
module top(
    input  i_rx,
    input  clk,
    output o_tx,
    input  rst
);

    // UART signals
    reg    tx_start;
    wire   rx_valid, tx_done, tx_busy;
    reg [127:0] tx_byte;
    wire [127:0] rx_byte;

    // UART module
    uart_sm #(.BAUD(115200), .CLK_SPEED(12_000_000)) UART_LINK (
        .clk(clk),
        .reset(rst),
        .i_rx(i_rx),
        .o_tx(o_tx),
        .tx_start(tx_start),
        .tx_done(tx_done),
        .tx_busy(tx_busy),
        .rx_done_really(rx_valid),
        .rx_words(rx_byte),
        .tx_words(tx_byte)
    );

    // FSM states
    localparam RST    = 3'b000,
               WAIT    = 3'b001,
               COMPUTE = 3'b010,
               START_TX = 3'b011,
               WAIT_TX  = 3'b100,
               WAIT2    = 3'b101;

    reg [2:0] state;
    reg [127:0] byte;

    wire [127:0] sub_bytes_out;
```

```

sub_bytes u_sub_bytes (
    .in(byte),
    .out(sub_bytes_out)
);

wire [127:0] shift_rows_out;

shift_rows u_shift_rows (
    .in(sub_bytes_out),
    .out(shift_rows_out)
);

wire [127:0] mc_out;

mix_columns u_mix_columns (
    .in(shift_rows_out),
    .out(mc_out)
);

// --- State Machine ---
always @(posedge clk) begin
    if (rst) begin
        state    <= RST;
        byte     <= 128'b0;
        tx_byte  <= 128'b0;
        tx_start <= 1'b0;
    end else begin
        case (state)
            RST: begin
                state <= WAIT;
            end

            WAIT: begin
                if (rx_valid) begin
                    byte <= rx_byte;
                    state <= WAIT2;
                end
            end

            WAIT2: begin
                state <= COMPUTE;
            end
        endcase
    end
end

```

```

    COMPUTE: begin
        tx_byte <= mc_out; // Send the full output
        state <= START_TX;
    end

    START_TX: begin
        tx_start <= 1'b1;
        state <= WAIT_TX;
    end

    WAIT_TX: begin
        tx_start <= 1'b0;
        if (tx_done)
            state <= RST;
        end
    endcase
end
end
endmodule

```

```

`timescale 1ns/1ps

```

```

module tb_shift_rows;
    reg [127:0] in;
    wire [127:0] out;

```

```

// DUT
shift_rows dut (
    .in(in),
    .out(out)
);

function [127:0] ref_shift_rows;
    input [127:0] x;
    reg [7:0] s [0:15];
    begin
        { s[0], s[1], s[2], s[3],
          s[4], s[5], s[6], s[7],
          s[8], s[9], s[10], s[11],
          s[12], s[13], s[14], s[15] } = x;

        ref_shift_rows = {
            s[0], s[4], s[8], s[12],
            s[5], s[9], s[13], s[1],
            s[10], s[14], s[2], s[6],
            s[15], s[3], s[7], s[11]
        };
    end
endfunction

integer i;
reg [127:0] exp;

initial begin
    // Test 1: Identity-friendly ramp bytes 0x00..0x0F placed as {b0..b15}
    in = 128'h00_01_02_03_04_05_06_07_08_09_0A_0B_0C_0D_0E_0F;
    #1;
    exp = ref_shift_rows(in);
    if (out !== exp) begin
        $display("FAIL T1: out=%032h exp=%032h", out, exp);
        $fatal;
    end else begin
        $display("PASS T1: out=%032h", out);
    end

    // Test 2: Random-looking pattern
    in = 128'hD4_E0_B8_1E_BF_B4_41_27_5D_52_11_98_30_AE_F1_E5;
    #1;
    exp = ref_shift_rows(in);
    if (out !== exp) begin

```

```

        $display("FAIL T2: out=%032h exp=%032h", out, exp);
        $fatal;
    end else begin
        $display("PASS T2: out=%032h", out);
    end

    // Test 3: All same byte
    in = {16{8'hAA}};
    #1;
    exp = ref_shift_rows(in);
    if (out !== exp) begin
        $display("FAIL T3: out=%032h exp=%032h", out, exp);
        $fatal;
    end else begin
        $display("PASS T3: out=%032h", out);
    end

    $display("All shift_rows tests passed.");
    $finish;
end
endmodule

```

```

`timescale 1ns/1ps

module tb_mix_column;
    reg [31:0] in_col;
    wire [31:0] out_col;

    // DUT
    mix_column dut (

```

```

.in(in_col),
.out(out_col)
);

// Helper: pretty print a 32-bit column as four bytes
task show_vec;
    input [31:0] v;
    begin
        $display("%02h %02h %02h %02h", v[31:24], v[23:16], v[15:8], v[7:0]);
    end
endtask

// Known-good GF(2^8) MixColumns example from FIPS-197:
// Input column [db 13 53 45] -> Output [8e 4d a1 bc]
// Note: The module declares in as {a0,a1,a2,a3} where a0 is the MSB byte.
// We'll supply in_col = 32'hdb_13_53_45 and expect 32'h8e_4d_a1_bc.
initial begin
    // Test 1: FIPS-197 example 1
    in_col = 32'hdb_13_53_45;
    #1;
    if (out_col !== 32'h8e_4d_a1_bc) begin
        $display("FAIL T1: got=");
        show_vec(out_col);
        $display(" exp=8e 4d a1 bc");
        $fatal;
    end else begin
        $display("PASS T1: FIPS column ok");
    end

    // Test 2: All ones -> remains all ones
    // Matrix*[01 01 01 01]^T = [01 01 01 01]^T
    in_col = {4{8'h01}}; // 32'h01_01_01_01
    #1;
    if (out_col !== {4{8'h01}}) begin
        $display("FAIL T2: got=");
        show_vec(out_col);
        $display(" exp=01 01 01 01");
        $fatal;
    end else begin
        $display("PASS T2: all-ones column ok");
    end

    // Test 3: All zeros -> all zeros
    in_col = 32'h00_00_00_00;

```

```

#1;
if (out_col !== 32'h00_00_00_00) begin
    $display("FAIL T3");
    $fatal;
end else begin
    $display("PASS T3: all-zeros column ok");
end

$display("All mix_column tests passed.");
$finish;
end
endmodule

```

```

`timescale 1ns/1ps
// -----
// DUT wrapper: one AES "round core" without AddRoundKey
// state_out = MixColumns( ShiftRows( SubBytes(state_in) ) )
// -----
module round_chain (
    input wire [127:0] state_in,
    output wire [127:0] state_out
);
    // SubBytes: 16 parallel S-boxes, byte-wise
    wire [127:0] subbytes_out;
    genvar i;
    generate
        for (i = 0; i < 16; i = i + 1) begin : G_SBOX
            wire [7:0] in_b = state_in[127 - 8*i -: 8];
            wire [7:0] out_b;
            s_box u_sbox (.in_byte(in_b), .out_byte(out_b));
            assign subbytes_out[127 - 8*i -: 8] = out_b;
        end
    end
end

```

```

endgenerate

// ShiftRows
wire [127:0] sr_out;
shift_rows U_SR (.in(subbytes_out), .out(sr_out));

// MixColumns (128-bit variant that applies mix_column per 32-bit column)
wire [127:0] mc_out;
mix_columns U_MC (.in(sr_out), .out(mc_out));

    assign state_out = mc_out;
endmodule

// -----
// Testbench
// -----
module tb_all_together;
    // No clock required: the chain is purely combinational
    reg [127:0] in_state;
    wire [127:0] out_state;

    // DUT: only the three transforms
    round_chain DUT (
        .state_in(in_state),
        .state_out(out_state)
    );

    // Pretty-print a 128-bit state as 16 bytes
    task show_state;
        input [127:0] s;
        begin
            $display("%02h %02h %02h %02h %02h %02h %02h %02h %02h %02h %02h %02h %02h %02h %02h %02h",
                s[127:120], s[119:112], s[111:104], s[103:96],
                s[95:88], s[87:80], s[79:72], s[71:64],
                s[63:56], s[55:48], s[47:40], s[39:32],
                s[31:24], s[23:16], s[15:8], s[7:0]);
        end
    endtask

    // Simple "not-X" check
    task assert_no_x;
        input [127:0] v;
        input [127:0] exp_mask;

```

```

begin
    if ((^v) === 1'bx) begin
        $display("FAIL: X detected in vector!");
        $fatal;
    end
end
endtask

initial begin
    $display("==== tb_top (SubBytes -> ShiftRows -> MixColumns) ===");

    // Test 1: byte ramp
    in_state = 128'h00_01_02_03_04_05_06_07_08_09_0A_0B_0C_0D_0E_0F;
    #1;
    $display("T1 in :"); show_state(in_state);
    $display("T1 out:"); show_state(out_state);
    assert_no_x(out_state, 128'hFFFF_FFFF_FFFF_FFFF_FFFF_FFFF_FFFF_FFFF);

    // Test 2: FIPS-like pattern (used in many AES examples)
    in_state = 128'hD4_E0_B8_1E_BF_B4_41_27_5D_52_11_98_30_AE_F1_E5;
    #1;
    $display("T2 in :"); show_state(in_state);
    $display("T2 out:"); show_state(out_state);
    assert_no_x(out_state, 128'hFFFF_FFFF_FFFF_FFFF_FFFF_FFFF_FFFF_FFFF);

    // Test 3: all equal bytes
    in_state = {16{8'hAA}};
    #1;
    $display("T3 in :"); show_state(in_state);
    $display("T3 out:"); show_state(out_state);
    assert_no_x(out_state, 128'hFFFF_FFFF_FFFF_FFFF_FFFF_FFFF_FFFF_FFFF);

    $display("tb_top completed (visual check).");
    $finish;
end
endmodule

```

```
import sys
import time
import serial
import random
```

```
SERIAL_PORT = "COM3" #check device manager and change to that com port
BAUD_RATE = 115200
NUM_TESTS = 20
```

```
SBOX = [
0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x76,
0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0,
0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x31,0x15,
0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75,
0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84,
0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf,
0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8,
0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2,
0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73,
0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb,
0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79,
0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08,
0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a,
0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0x0e,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e,
0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf,
0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16
]
```

```
def sub_bytes(state_bytes):
    return bytes(SBOX[b] for b in state_bytes)
```

```
def shift_rows(state_bytes):
    s = list(state_bytes)
    m = [
        [s[0], s[4], s[8], s[12]], # row 0
        [s[1], s[5], s[9], s[13]], # row 1
        [s[2], s[6], s[10], s[14]], # row 2
        [s[3], s[7], s[11], s[15]], # row 3
    ]
    out = []
```

```

for i, row in enumerate(m):
    out.extend(row[i:] + row[:i])
return bytes(out)

```

```

def mul(a, b):
    res = 0
    for _ in range(8):
        if b & 1:
            res ^= a
        high = a & 0x80
        a = (a << 1) & 0xFF
        if high:
            a ^= 0x1B
        b >>= 1
    return res

```

```

def mix_single_column(col):
    a = list(col)
    return bytes([
        mul(2,a[0]) ^ mul(3,a[1]) ^ a[2] ^ a[3],
        a[0] ^ mul(2,a[1]) ^ mul(3,a[2]) ^ a[3],
        a[0] ^ a[1] ^ mul(2,a[2]) ^ mul(3,a[3]),
        mul(3,a[0]) ^ a[1] ^ a[2] ^ mul(2,a[3])
    ])

```

```

def mix_columns(state):
    s = [list(state[i::4]) for i in range(4)]
    mixed = [mix_single_column(col) for col in s]
    out = bytearray(16)
    for c in range(4):
        for r in range(4):
            out[r + 4*c] = mixed[c][r]
    return bytes(out)

```

```

def send_and_receive(port, baud, data, timeout=1.0):
    with serial.Serial(port, baud, timeout=timeout) as ser:
        ser.reset_input_buffer()
        ser.reset_output_buffer()
        time.sleep(0.02)
        ser.write(data)
        ser.flush()
        time.sleep(0.02)
        rx = ser.read(16)
    return rx

```

```

def aes_sub_shift_mix(state):
    return mix_columns(shift_rows(sub_bytes(state)))

# --- Main test ---
def main():
    print(f"Testing SubBytes -> ShiftRows -> MixColumns chain on {SERIAL_PORT} @
{BAUD_RATE}")
    total = 0
    passed = 0

    for i in range(NUM_TESTS):
        #random 16-byte input
        input_state = bytes(random.getrandbits(8) for _ in range(16))

        #expected reference output
        expected = aes_sub_shift_mix(input_state)

        rx = send_and_receive(SERIAL_PORT, BAUD_RATE, input_state)

        ok = (len(rx) == 16 and rx == expected)
        print(f"\nTest {i+1:02d}")
        print(f" Input   : {input_state.hex()}")
        print(f" Expected : {expected.hex()}")
        if len(rx) != 16:
            print(f" Received : (incomplete, {len(rx)} bytes) {rx.hex()}")
        else:
            print(f" Received : {rx.hex()}")
        print(" → PASS" if ok else " → FAIL")

        total += 1
        passed += ok

    print(f"\nSummary: {passed}/{total} passed ({100*passed/total:.1f}%)")

if __name__ == "__main__":
    main()

```

```
Expected : c9ec78e676e0a8702af66d34a1993141
Received : c9ec78e676e0a8702af66d34a1993141
→ PASS
```

Test 15

```
Input      : f594e3c48f9facc17e13b99e372f70bd
Expected   : 8dcb1f482c668881ab924cfad37d9c10
Received   : 8dcb1f482c668881ab924cfad37d9c10
→ PASS
```

Test 16

```
Input      : eebf95d7f4107ff1407720894d4c5048
Expected   : f03761a13cb5fe60e2848c41b039a6b1
Received   : f03761a13cb5fe60e2848c41b039a6b1
→ PASS
```

Test 17

```
Input      : 6171237d18191556d0ecfafe082676f5
Expected   : 69cd5004cf9d09ff755e03387c3df7c7
Received   : 69cd5004cf9d09ff755e03387c3df7c7
→ PASS
```

Test 18

```
Input      : 63ed21ade9a9ce62a69c96eb38047286
Expected   : 57a9dfdd90ece48d126dd22c93c27f1e
Received   : 57a9dfdd90ece48d126dd22c93c27f1e
→ PASS
```

Test 19

```
Input      : 478c209dba1087497b0004f374b620ce
Expected   : 67a913cebfae006f1c4457ec896ecfc4
Received   : 67a913cebfae006f1c4457ec896ecfc4
→ PASS
```

Test 20

```
Input      : 5ec6a3f32da26ee7892b87abb8773e82
Expected   : fa0679e31ce1412acfdce13ee2c75b5b
Received   : fa0679e31ce1412acfdce13ee2c75b5b
→ PASS
```

Summary: 20/20 passed (100.0%)

PS C:\Users\david\Documents\AES Project\part1> █